# Machine Learning un prognozēšanas modeļi

3. nedēļa: Regresijas pārskats, klasterizācija + validācija

Finansē Eiropas Savienība NextGenerationEU

2027 Nacionālais attīstības plāns

Projekts "Digitālo tehnoloģiju un procesu pārvaldības prasmju uzlabošana Latvijas uzņēmumos"
Nr. 2.3.1.2.i.0/2/24/A/CFLA/007

FITA

# Atceramies: 2 nedēļas ML pamatiem

> *"Jūs jau zināt ML fundamentus. Šonedēļ ejam tālāk."*

1. ML ievads, datu sagatavošana, klasifikācija (pabeigts)
2. Klasifikācijas pārskats, regresija (pabeigts)
3. **Regresijas pārskats, klasterizācija + validācija (aktīvā)**
4. Pārskats, gala projekts
5. Gala projekta prezentācijas

# 3. nedēļas plāns

Daļa 1
**30%**

**Regresijas atkārtojums**
rezultāti, problēmas, optimizācija

Daļa 2
**35%**

**Klasterizācija**
unsupervised learning, K-Means, interpretācija

Daļa 3
**20%**

**Modeļu validācija**
Cross-validation, Pipeline, GridSearchCV

Daļa 4
**15%**

**W3 mājas darbs**
kombinēts uzdevums (klasterizācija + validācija)

**UZMANĪBU: Šonedēļ ir PAŠAS SMAGĀKĀ NEDĒĻA! Mājas darbam plānojiet 120-240 min un sāciet tūlīt.**

# Daļa 1: Regresijas pārskats

*Kā jums klājās pagājušajā nedēļā?*

# Regresija: ātrs konspekts

| Ko mēs darījām | Kāpēc tas svarīgi? |
|---|---|
| **Lineāra regresija** | Vienkāršs bāzlīnijas modelis, interpretējams |
| **Polynomiskā regresija** | Iegūst nelineāras sakarības |
| **Ridge/Lasso** | Novērš overfitting, mazina iezīmes |
| **RandomForest regresija** | Precīzāka, bez manual feature engineering |
| **Train/Test split** | Novērtē reālus modeļa rezultātus |

**Mērķis:** Prognozēt skaitliskas vērtības (EUR, laiks, skaits) ar zemu kļūdu skaitu.

# Regresijas metrikas: pieņēmumi

| Metrika | Ko tas nozīmē? | Labs rezultāts |
|---|---|---|
| MAE | Vidējā absolūtā kļūda | Mazāks = labāks |
| RMSE | Kvadrātiskā kļūda — soda lielās kļūdas | Mazāks = labāks |
| $R^2$ | % no variācijas, ko modelis skaidro | Lielāks = labāks (0-1) |

**Kritisks jautājums:**
Jūsu MAE ir 5 EUR, $R^2 = 0.72$. Vai tas ir labs? Depends on business! 5 EUR kļūda ir labā for price prediction, bet sliktā for medicine dose.

# Padomā par to

**Jūs trenējāt 3 modeļus — kurš ir labākais?**

| Modelis | Train MAE | Test MAE | Train $R^2$ | Test $R^2$ |
|---|---|---|---|---|
| Lineāra | 6.5 | 7.2 | 0.64 | 0.62 |
| Polynomial | **2.1** | **8.9** | **0.95** | **0.48** |
| **RandomForest** | 5.1 | **6.8** | 0.78 | **0.70** |

**Kāpēc Polynomial izskatās "vislabāk" uz training set, bet neveicas test set?**

**OVERFITTING —** polynomālais modelis iegaumēja training datus, nevis iemācījās vispārējus modeļus.

# Biežākās regresijas problēmas

1. **Data leakage** — mērogošana PIRMS split
2. **Outliers** — viena liela kļūda “nobeidz" modeli
3. **Feature scaling** — features ar dažādiem lielumiem dominē
4. **Underfitting** — pārāk vienkāršs modelis
5. **Overfitting** — pārāk sarežģīts modelis

## Šonedēļ apgūsim:

**Pipeline**
= novērš data leakage

**Cross-validation**
= patiesa F1 / MAE / $R^2$

**GridSearchCV**
= automātiska parametru optimizācija

# Daļa 2: Klasterizācija

*Unsupervised Learning — grupēšana bez atbildēm*

# Pirks vai nepirks — zinām.
# Cik iztērēs — zinām.
# Bet KĀDI ir mūsu klienti?

> *"Labākais veids, kā saprast savus klientus, ir ļaut datiem pašiem pastāstīt."*

**Klasterizācija atbild uz:** Ko, ja nav pareizo atbilžu? Kā atrast dabiskās grupas datos?

# Supervised vs. Unsupervised

**Klasifikācija (W1-2)**
Mums IR etiķetes (pirks/nepirks) --> modelis mācās no tām

**Regresija (W2)**
Mums IR pareizās atbildes (EUR summas) --> modelis mācās no tām

**Klasterizācija (W3)**
Mums NAV etiķešu! --> modelis PATS atrod grupas

**Supervised** = ir skolotājs, kas pārbauda atbildes. **Unsupervised** = nav skolotāja — dators pats atrod struktūru datos.

# Kas ir klasterizācija?

## Analoģija: Veļas šķirošana

Iedomājies, ka tev ir kalns ar veļu. Neviens nav teicis, kā to šķirot. Bet tu intuitīvi sadali pa krāsām, materiāliem, izmēriem.

**Tieši tā strādā klasterizācija — algoritms pats atrod līdzīgus objektus un sagrupē tos.**

Unsorted data — no instructions given.

Nobody told you HOW to sort — you found natural groups based on similarity. That's clustering.

**Clustering** = unsupervised learning uzdevums, kur algoritms sadala datus grupās (klasteros) pēc līdzības, bez iepriekš zināmām kategorijām.

# Klasterizācijas piemēri reālajā dzīvē

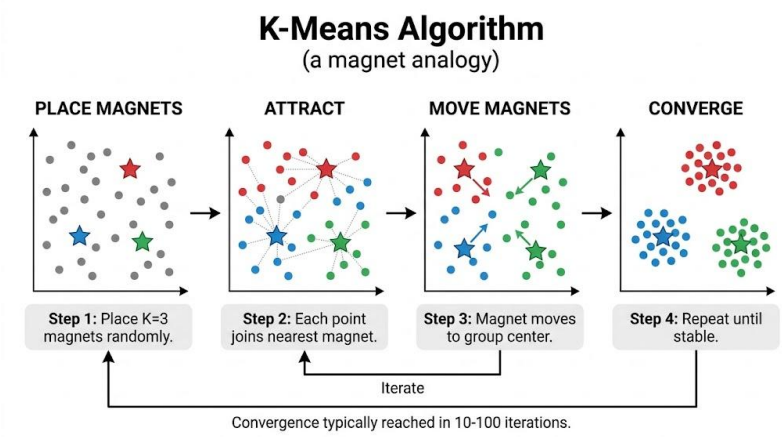
| Problēma | Ko grupē? | Kāpēc? |
|---|---|---|
| Klientu segmentācija | Klientus pēc uzvedības | Personalizēts mārketings |
| Dokumentu grupēšana | Tekstus pēc tēmas | Automātiska organizēšana |
| Attēlu saspiešana | Pikseļus pēc krāsas | Faila izmēra samazināšana |
| Anomāliju detektēšana | "Normālās" grupas | Atklāt neparastus gadījumus |
| **DataShop** | **Klientus pēc pirkumu uzvedības** | **Pielāgot piedāvājumus** |

Visur, kur vēlies atrast dabiskas grupas datos bez iepriekš zināmām kategorijām — tā ir klasterizācijas vieta.

# K-Means: "Magnētu analoģija"

Iedomājies: Tu izmet uz galda K magnētus starp datu punktiem.

1. Katrs punkts "pieķeras" tuvākajam magnētam
2. Magnēts pārvietojas uz grupas centru
3. Punkti pārķeras no jauna
4. Atkārto, līdz nekas nemainās




**K-Means** = izvieto K centrus (centroīdus), katru punktu piešķir tuvākajam centram, pārvieto centru uz grupas viduspunktu, atkārto.

# K-Means algoritms: soli pa solim

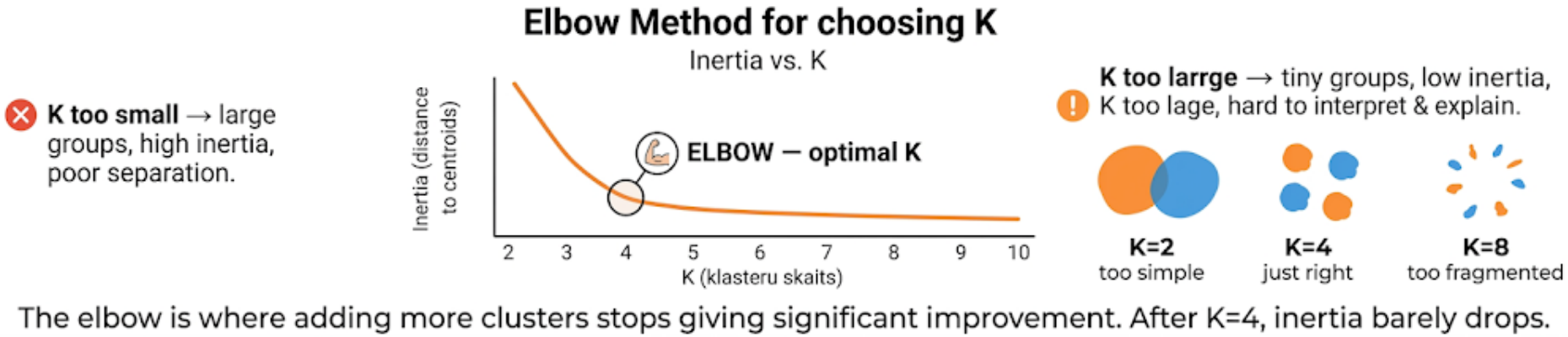
1. **Izvēlies K** — cik grupas vēlies (piem. K=3)
2. **Novieto K centroīdus** — izlases pozīcijās
3. **Piešķir katru punktu** — tuvākajam centroīdam
4. **Pārvieto centroīdu** — uz grupas viduspunktu
5. **Atkārto soļus 3-4** — līdz centroīdi vairs nepārvietojas

**Svarīgi:**
K ir jāizvēlas tev — algoritms to nezina! Sākuma pozīcija ir nejauša --> random_state fiksē rezultātu.

# Elbow metode: kā izvēlēties K?

**Problēma:** Cik grupu ir pareizi? K=2? K=5? K=20?

**Risinājums:** Izmēģini daudz K vērtību un mēri inertia (cik cieši punkti ir pie sava centroīda).




**Elbow method** = "Dilstošā atdeve" — sākotnēji katrs papildus klasteris daudz uzlabo, bet pēc kāda brīža uzlabojums kļūst minimāls. "Elkonis" ir vieta, kur pārtraukt.

# K-Means kodā

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Obligāti mērogot features pirms klasterizācijas!
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# K-Means ar 3 klasteriem
model = KMeans(n_clusters=3, random_state=42)
clusters = model.fit_predict(X_scaled)
```

**Jauns: `fit_predict()`** = iemācās un uzreiz piešķir klasterus. Nav atsevišķa predict() soļa!

# Klasteru vizualizācija ar PCA

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.scatter(X_pca[:, 0], X_pca[:, 1],
            c=clusters, cmap='viridis', alpha=0.6)
plt.scatter(pca.transform(model.cluster_centers_)[:, 0],
            pca.transform(model.cluster_centers_)[:, 1],
            marker='X', s=200, c='red')
plt.title('Klientu klasteri (PCA vizualizācija)')
```

**PCA** samazina dimensijas uz 2D vizualizācijai. Katrs punkts = klients, krāsa = klasteris, sarkanie krustiņi = centroīdi.

# Centroīdu analīze: kas definē katru grupu?

```
centers = pd.DataFrame(
    scaler.inverse_transform(model.cluster_centers_),
    columns=feature_names
)
print(centers)
```

**Piemēra izvade:**

| Klasteris | Apmeklējumi | Laiks lapā | Preces | Vid. summa |
|---|---|---|---|---|
| 0 | 3 | 4 min | 2 | 18 EUR |
| 1 | 15 | 22 min | 8 | 85 EUR |
| 2 | 9 | 12 min | 5 | 42 EUR |

> **`scaler.inverse_transform()`** pārvērš centroīdus atpakaļ uz oriģinālajām mērvienībām.

# Klasteru nosaukumi: kur ML satiek biznesu

| Klasteris | Raksturojums | Biznesa nosaukums |
|---|---|---|
| 0 | Maz apmeklējumu, zems tēriņš | **"Jaunie klienti"** |
| 1 | Daudz apmeklējumu, augsts tēriņš | **"Lojālie klienti"** |
| 2 | Vidēji apmeklējumi, vidējs tēriņš | **"Izpētītāji"** |

**Algoritms dod skaitļus (0, 1, 2). JŪSU darbs ir dot tiem NOSAUKUMUS un NOZĪMI.** Tieši šeit ML kļūst par biznesa rīku.

# Daļa 3: Modeļu validācija

*Kā zināt, cik labs ir tavs modelis?*

# Problēma: viens split nav pietiekams

**Scenārijs 1:** `random_state=42` --> Test F1 = **0.72**

**Scenārijs 2:** `random_state=123` --> Test F1 = **0.68**

**Scenārijs 3:** `random_state=999` --> Test F1 = **0.75**

## Kurš rezultāts ir PATIESS?

**ATBILDE:** Tas ir VIENĪGS viena split variants. Viens split = vienas nejaušības lēmums. Tu nevari zināt, cik uzticams ir rezultāts bez vairākām pārbaudēm.

# Cross-validation: labākais risinājums

**Ideja:** Neveic VIENU split. Veic PIECUS split un iemācies no katra!

```
Dati (100 rindas)
│
├─ Eksperiments 1: Fold 1 test, Folds 2-5 train -> F1 = 0.72
├─ Eksperiments 2: Fold 2 test, Folds 1,3-5 train -> F1 = 0.75
├─ Eksperiments 3: Fold 3 test, Folds 1-2,4-5 train -> F1 = 0.71
├─ Eksperiments 4: Fold 4 test, Folds 1-3,5 train -> F1 = 0.74
└─ Eksperiments 5: Fold 5 test, Folds 1-4 train -> F1 = 0.73

Vidējais F1: (0.72 + 0.75 + 0.71 + 0.74 + 0.73) / 5 = 0.73 ± 0.01
```

**5-fold cross-validation** = iegūst 5 dažādus modeļa rezultātus un vidējais ir DAUDZ ticamāks.

# Pipeline: novēršam datu noplūdi

**NEPAREIZI — data leakage!**

```
scaler.fit_transform(X)  # VISUS!
X_train, X_test = train_test_split
model.fit(X_train, y_train)
```

**PAREIZI — Pipeline**

```
from sklearn.pipeline import
make_pipeline

pipe = make_pipeline(
    StandardScaler(),
    RandomForestClassifier()
)
```

**Data leakage** = skalers jau "zina" test datus pirms split notiek. Rezultāts ir KĻŪDAINS. Pipeline nodrošina, ka skalers fit TIKAI uz training datiem.

# GridSearchCV: automātiska parametru optimizācija

```
param_grid = {
    'randomforestclassifier__n_estimators': [50, 100, 200],
    'randomforestclassifier__max_depth': [5, 10, 20]
}

grid = GridSearchCV(pipe, param_grid, cv=3, scoring='f1')
grid.fit(X_train, y_train)

print(grid.best_params_)  # Labākie parametri
print(grid.best_score_)   # Labākais CV F1
```

**GridSearchCV** = mēģina visas 9 kombinācijas (3x3) un automātiski ATROD labākos parametrus ar cross-validation.

# Validācijas workflow: kopsavilkums

## Viena split vietā:

1. Izvēlies Pipeline
   (StandardScaler + Model)
2. Uztrenē ar cross-validation (cv=5)
3. Iegūsti vidējo F1 / MAE / $R^2$
4. Zini, cik noturīgs ir modelis

## Manuālas meklēšanas vietā:

1. Nosaki GridSearchCV
   ar parameter grid
2. Izpildi (pārbaudīs visu ar CV)
3. Iegūsti labākos parametrus
4. Zini, ka OPTIMIZĒJĀT pareizi

# Daļa 4: W3 mājas darbs

*Kombinēts klasterizācijas + validācijas uzdevums*

# W3 Mājas darbam: kas tevi gaida?

**UZMANĪBU: ŠIS IR PATS SMAGĀKAIS DARBS KURSĀ!**

**Part A:** Klasterizācija (K-Means, Elbow, PCA, interpretācija)

**Part B:** Modeļu validācija (CV, Pipeline, GridSearchCV)

**Ilgums:** ~120-240 minūtes

**Fails:** `week3/week3_homework.ipynb`

**Sāc TŪLĪT.**

# Part A: Klasterizācija (7 uzdevumi)

1. Datu sagatavošana — ielādē, izvēlies 3-5 features, normalizē
2. Elbow metode — atrod optimālo K (diapazons 2-10)
3. KMeans modelis — uztrenē ar izvēlēto K
4. PCA vizualizācija — parādi klasterus 2D scatter plot
5. Centroīdu analīze — apraksti katru grupu ar skaitļiem
6. Klasteru heatmap — vizuāli profili
7. Biznesa interpretācija — nosaukumi un rekomendācijas

> StandardScaler PIRMS KMeans! Elbow grafiks. Markdown šūna par interpretāciju.

# Part B: Modeļu validācija (5 uzdevumi)

1. Cross-validation — 5-fold CV uz RandomForest, salīdzini vienu split vs. CV
2. Pipeline + data leakage — parādi problēmu un risinājumu
3. GridSearchCV — optimizē 2+ parametrus, parādi labāko kombināciju
4. Feature importance — top 10 svarīgākās iezīmes
5. Modeļu salīdzinājums — RandomForest vs. XGBoost ar CV

**Hints:** cross_val_score(cv=5). Pipeline(StandardScaler, model). GridSearchCV ar heatmap.

# Ieteicamā darba secība

**1. diena:** Izlasi visu materiālu + meklē kodus tiešsaistē

**2. diena:** Sāc Part A (klasterizācija)

**3. diena:** Pabeidz A, sāc Part B

**4. diena:** Izpildi Part B, pārbaudi

**5. diena:** Finalizē, iesniedz, atsāc W4

**SVARĪGI:** Pirms iesniegšanas: Kernel --> Restart & Run All un pārliecinies, ka visas šūnas strādā.

# Nākamā nedēļa: Pārskats + gala projekts

## 4. nedēļa

- Mājas darba pārskats
- Gala projekta plānošana

(dataset, problēma, ML tips, metrikas)

- Gala projekta sāknesis

## 5. nedēļa

- Gala projekta finalizēšana
- Prezentācijas

**W3 ir fundaments. W4-5 jūs veiksiet izpēti un prezentēsiet rezultātus.**

# Paldies!

## Veiksmi W3!

Atcerieties: dati ir visur. ML ir rīks, nevis risinājums, kas strādās mūžīgi.

*"Labākais modelis nav tas, kas ir precīzākais uz testa datiem.*
*Labākais modelis ir tas, kas palīdz pieņemt labākus biznesa lēmumus."*